

IIT: Accurate Decentralized Application Identification Through Mining Intra- and Inter-Flow Relationships

Qianwei Meng[✉], Qingjun Yuan[✉], Weina Niu[✉], *Senior Member, IEEE*, Yongjuan Wang[✉],
Siqi Lu[✉], Guangsong Li, Xiangbin Wang, and Wenqi He

Abstract—Identifying Decentralized Applications (DApps) from encrypted network traffic plays an important role in areas such as network management and threat detection. However, DApps deployed on the same platform use the same encryption settings, resulting in DApps generating encrypted traffic with great similarity. In addition, existing flow-based methods only consider each flow as an isolated individual and feed it sequentially into the neural network for feature extraction, ignoring other rich information introduced between flows, and therefore the relationship between different flows is not effectively utilized. In this study, we propose a novel encrypted traffic classification model IIT to heterogeneously mine the potential features of intra- and inter-flows, which contain two types of encoders based on the multi-head self-attention mechanism. By combining the complementary intra- and inter-flow perspectives, the entire process of information flow can be more completely understood and described. IIT provides a more complete perspective on network flows, with the intra-flow perspective focusing on information transfer between different packets within a flow, and the inter-flow perspective placing more emphasis on information interaction between different flows. We captured 44 classes of DApps in the real world and evaluated the IIT model on two datasets, including DApps and malicious traffic classification tasks. The results demonstrate that the IIT model achieves a classification accuracy of greater than 97% on the real-world dataset of 44 DApps, outperforming other state-of-the-art methods. In addition, the IIT model exhibits good generalization in the malicious traffic classification task.

Index Terms—Decentralized applications, encrypted traffic, blockchain, transformer, deep learning.

Received 27 March 2024; revised 9 July 2024; accepted 9 October 2024. Date of publication 11 October 2024; date of current version 14 March 2025. This work is supported by The National Key Research and Development Program of China No.2023YFB2705000 and National Natural Science Foundation of China 62276091. The associate editor coordinating the review of this article and approving it for publication was L. Cui. (Corresponding author: Qingjun Yuan.)

Qianwei Meng, Yongjuan Wang, Siqi Lu, Guangsong Li, Xiangbin Wang, and Wenqi He are with the Henan Key Laboratory of Network Cryptography Technology and the Key Laboratory of Cyberspace Security, Ministry of Education, Zhengzhou 450001, China (e-mail: mengqw20@163.com; pinkywyj@163.com; 080lusiqi@sina.com; lgsok@163.com; Moskys@outlook.com; hewenqixd@163.com).

Qingjun Yuan is with the MoE Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an 710049, China, and also with the Henan Key Laboratory of Network Cryptography Technology and the Key Laboratory of Cyberspace Security, Ministry of Education, Zhengzhou 450001, China (e-mail: gcxyuan@outlook.com).

Weina Niu is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 610056, China (e-mail: niuweina1@126.com).

Digital Object Identifier 10.1109/TNSM.2024.3479150

I. INTRODUCTION

BLOCKCHAIN technology has rapidly developed in recent years because of its decentralization, anonymity and auditability [1]. This has also brought about the rise of decentralized applications (DApps). However, malicious developers may create seemingly legitimate DApps but plant fraudulent or scammy behavior behind them. Surprisingly, finance-related DApps account for more than 70% of the total number of DApps, and there are DApps in these categories that contain unfair rules or constitute Ponzi schemes or financial fraud. As a result, DApps have been subjected to a variety of attacks by cybercriminals for profit [2], and many high-impact real-world attacks against DApps have caused losses of millions of dollars [3]. Therefore, in order to effectively achieve network management and threat detection for DApps, it is crucial to classify DApps encrypted traffic from the traffic aspect.

For this purpose, various methods for encrypted traffic classification have been proposed [4]. They can be broadly grouped into two categories: **Intra-flow** based methods and **Inter-flow** based methods. These two categories of methods deal with different objects. Intra-flow based methods focus on mining intra-flow features, while inter-flow based methods start to consider utilizing inter-flow features.

In general, to efficiently represent a flow, many previous studies used hand-designed features as **Intra-flow** features. Typically, these are statistical features [5], [6], [7] that are subsequently fed into machine learning algorithms. In deep-learning-based methods, no matter what form a flow is represented in, such as a feature vector or a graph, each flow is treated as an isolated individual and sequentially fed into the neural network for feature extraction [8], [9]. Therefore, the relationships between flows are not effectively utilized.

In contrast, **Inter-flow** based methods place greater focus on underutilized inter-flow features. For example, MT-FlowFormer utilizes the mechanism to extract deep-level features from the flow sequence, which can effectively distinguish the relevance and importance of different flows in the flow sequence [10]. Yao et al. and Vaswani et al. treats the stream sequence as a time sequence and used attention-based LSTM to learn the temporal relationships among network flows [11], [12]. However, the above methods have spatio-temporal limitations when dealing with flow sequences or time-induced flow segments [13].

To solve the above-mentioned problem, we design Intra- and Inter-flow Attention Blocks to heterogeneously mine intra- and inter-flow features, which are neglected by previously developed methods. Network flows provide hierarchical information about network events (intra-flow) and events in communication flows (inter-flow). The Intra-flow Attention Block can automatically consider the relationship between two arbitrary features within a flow. In contrast, the Inter-flow Attention Block can overcome spatial and temporal constraints to efficiently determine the correlation between any two different flows.

In this study, we design a novel network traffic classification method referred to as Intra-flow and Inter-flow Transformer (IIT). To achieve this, two types of side-channel information [14], namely the packet length sequence and packet arrival timestamp sequence, are used to represent a network flow. The model then maps each discrete value into a fixed dimensional vector through an embedding layer. Next, the Intra-flow Attention Block extracts the features for each flow, and the Inter-flow Attention Block learns the dependencies between flows. Ultimately, for each flow passing through the embedding and Attention modules, a set of distinguishing features is learned. Using weight adaptations via the self-attention mechanism, important features can be efficiently extracted and larger weights can be assigned to the more discriminative features, thereby realizing more effective feature extraction.

The main contributions of this study are summarized as follows:

- We proposed an end-to-end novel model, IIT, for DApps encrypted traffic classification, which can enhance the capability of DApps encrypted traffic classification by exploiting the intra-flow packet-to-packet and flow-to-flow relationships with the help of the self-attention mechanism.
- We designed Intra- and Inter-flow Attention Block to extract the features of intra- and inter-flow information in a heterogeneous manner to learn more representative flow features. This process avoids the influence of irrelevant features on the classification effect of the model.
- We constructed a dataset of 44 classes of DApps traffic on which model performance was evaluated. The proposed method outperformed other state-of-the-art methods, achieving a 97.164% accuracy. In addition, the model exhibited the best generalization performance on the task of classifying malicious traffic.

The remainder of this paper is organized as follows. The background and related work on DApp-encrypted network traffic classification and the focused problems are discussed in Section II. The proposed IIT model is introduced in detail in Section III. The proposed method and state-of-the-art baselines are analyzed and the experimental results are presented in Section IV. Finally, the paper is concluded in Section VI.

II. RELATED WORK

This study focused on the accurate classification of encrypted DApp traffic. Prior methods for encrypted traffic classification generally fall into two categories: Web/mobile

and DApp classification methods. Many studies have utilized machine- and deep-learning approaches to address the Web and mobile application-encrypted traffic classification problem. Here, as shown in Table I, only works that are closely related to this study will be reviewed.

A. Web and Mobile Application Classification Methods

Although an increasing number of applications are adopting SSL/TLS to protect their traffic, it is still possible to obtain large amounts of side-channel data information from encrypted traffic, such as packet lengths, directions, and TCP timestamps. Methods for classifying Web and mobile applications using side-channel information are the focus of this review.

Many statistical features can be obtained from the side-channel data sequences, including the maximum, minimum, mean, variance, skewness, kurtosis, and percentile. These statistical features can subsequently be used as inputs for machine- or deep-learning algorithms. AppScanner uses 54 statistical features of mobile app network flows as a Random Forest (RF) input to realize the real-time identification of mobile app network flows [5]. BIND extracts characteristics from encrypted traffic by utilizing the data dependencies that occur during the sequential transmissions of network packets [6]. Subsequently, RF was used to identify website and app fingerprinting. CUMUL abstracted the loading process of a webpage by generating a cumulative behavioral representation of its trace, for example, packet ordering or burst behavior, which was fed into an RF classifier [7].

Besides, deep learning has also been widely used for encrypted traffic classification. FS-Net was based on the encoding layer of a bidirectional gated recurrent unit (GRU) for automatic modeling of deep fingerprints in encrypted traffic sequences to improve the discriminative ability of the fingerprints [17]. Deep fingerprinting uses packet direction sequences as inputs to convolutional neural networks (CNNs) to identify traffic that accesses different websites via Tor [16]. ACID relies on low-dimensional embeddings trained using a lightweight neural model that is comprised of multiple kernel networks that optimally separate samples of different classes to classify malware traffic using an adaptive clustering network [19]. ProGraph is a graph-based approach that uses destination, certificate, and domain features to ensure robust network traffic classification in various network environments [22]. In addition to utilizing packet payloads, EBSNN uses side-channel information and a hierarchical attention network to learn high-level representations of packets to classify network traffic [25]. FA-Net adopts two hierarchical multi-head self-attention encoders to completely enumerate all potential intra-burst features and inter-burst dependencies to avoid heavy dependence on professional experience [26]. TFE-GNN fuses packet header and payload information to construct point-wise mutual information (PMI)-based graphs for stronger feature representation [8].

B. DApp Classification Methods

The aforementioned studies made important research contributions to the field of encrypted traffic classification.

TABLE I
THE COMPARISON WITH THE EXISTING METHODS

Model	Task	Classifier	Traffic Features	Advantages
Appscanner [5]	Web and Mobile Application Classification	SVM	Packet length sequence or statistical features	Lightweight framework
GraphDApp [26]	Decentralized Application Classification	GNN+MLP	Traffic interaction graph	Works well in both open and closed sets
DF [16]	Website Fingerprinting	CNN	Packet direction sequence	Sophisticated Convolutional Neural Networks
FS-Net [15]	Web and Mobile Application Classification	GRU+AE	Packet length sequence	The entire sequence information can be modelled to capture the contextual information in the encrypted flow
CUMUL [7]	Website and App Fingerprinting	RF	Packet size and packet direction	Abstracting the web page loading process by generating a cumulative behavioural representation of the web page traces
BIND [6]	Website and App Fingerprinting	RF	Sequences of packets in opposite directions	Superior to other methods in open world scenarios and when considering defence mechanisms
CQNet [22]	Decentralized Application Classification	CNN + Siamese Network	Payload of packet sequence	The use of a quaternion structure allows for more restrictive relationships between quaternions to be exploited and provides rich information to the classifier
ACID [17]	Malicious Traffic Classification	DNN+RF	Raw network packets	Introducing a novel supervised adaptive clustering technique to learn cluster centers
Var-CNN [27]	Website Fingerprinting	CNN	Packet direction, inter-packet time and metadata information	Designing a ResNet-based classifier that is effective in low-data scenarios
FG-Net [28]	Website and App Fingerprinting	GNN	Flow relationship graph	A novel flow-level relationship is utilised that is unique across applications and stable across application versions
Prograph [18]	Website and App Fingerprinting	GNN	Network sessions	Robust performance across a wide range of network environments
RoFi [29]	Website Fingerprinting	CNN	Traffic aggregation matrix	Attackers can fingerprint Tor traffic under various defences
CapsuleFormer [25]	Decentralized Application Classification	Capsule network	Statistical features	Fusing capsule networks and Transformer
This paper	Decentralized Application Classification	Transformer	Intra-flow and inter-flow features	Fully mine intra- and inter-flow features to achieve accurate and efficient classification of DApps.

Due to the fact that different classes of DApps show great similarity in packet lengths during the TLS handshake, the above methods perform poorly in solving DApps encrypted traffic classification. Moreover, these studies did not pay adequate attention to the network traffic generated by a large number of DApps on the Internet and the new challenges that DApp traffic brings to encrypted traffic classification.

In 2020, Wang et al. focused on the classification of DApp-encrypted traffic for three application scenarios: DApp, DApp user behavior, and general user behavior classifications [27]. In 2021, Wang et al. applied metric learning to DApp-encrypted traffic classification and proposed CQNet, which could easily filter out samples and learn more restrictive relationships using hard samples [18]. In 2019, Shen et al. used a feature selection function to generate statistical features from packet lengths, packet bursts, and time series [28]. They then filtered the features suitable for the fusion function and adopted the fused features in a machine-learning algorithm. In 2022, Hu et al. proposed an Ethereum traffic identification system in which a library of active nodes was built to filter potential Ethereum traffic [29]. Subsequently, a machine-learning classifier was used to further determine the potential Ethereum traffic based on the extracted Ethereum traffic features. In 2024, Zhou et al. proposed the CapsuleFormer model, which used capsule neurons to extract potential features from the encrypted traffic patterns of DApps [24].

C. Summary

There are two limitations to existing methods. First, the traditional machine-learning classifier approach relies on expert experience, the design of the classifier is task oriented, and classifier generalization is weak. Moreover, feature extraction from the model is time-consuming. Second, methods using deep neural networks typically use flows as inputs, and existing methods rarely comprehensively consider intra- and inter-flow relationships. Intra- and inter-flows represent different levels of information that should be processed heterogeneously.

III. METHODOLOGY

This section describes the design process of the IIT model in detail. A comprehensive overview of the IIT is presented in Fig. 1.

First, the encrypted DApp traffic is collected and the packet sequence is parsed into flows. The packet length, arrival time, and length cumulative sum (CS) sequences are employed to represent each network flow. Subsequently, the numerical features of each flow are mapped to fixed-dimensional vectors through an embedding layer. Second, the embedded network flow is fed into the Intra-flow Attention Block, which is the transformer's encoder. Each feature in the embedded network flow learns its relationship with the rest of the features and assigns a larger weight to the important features and a smaller

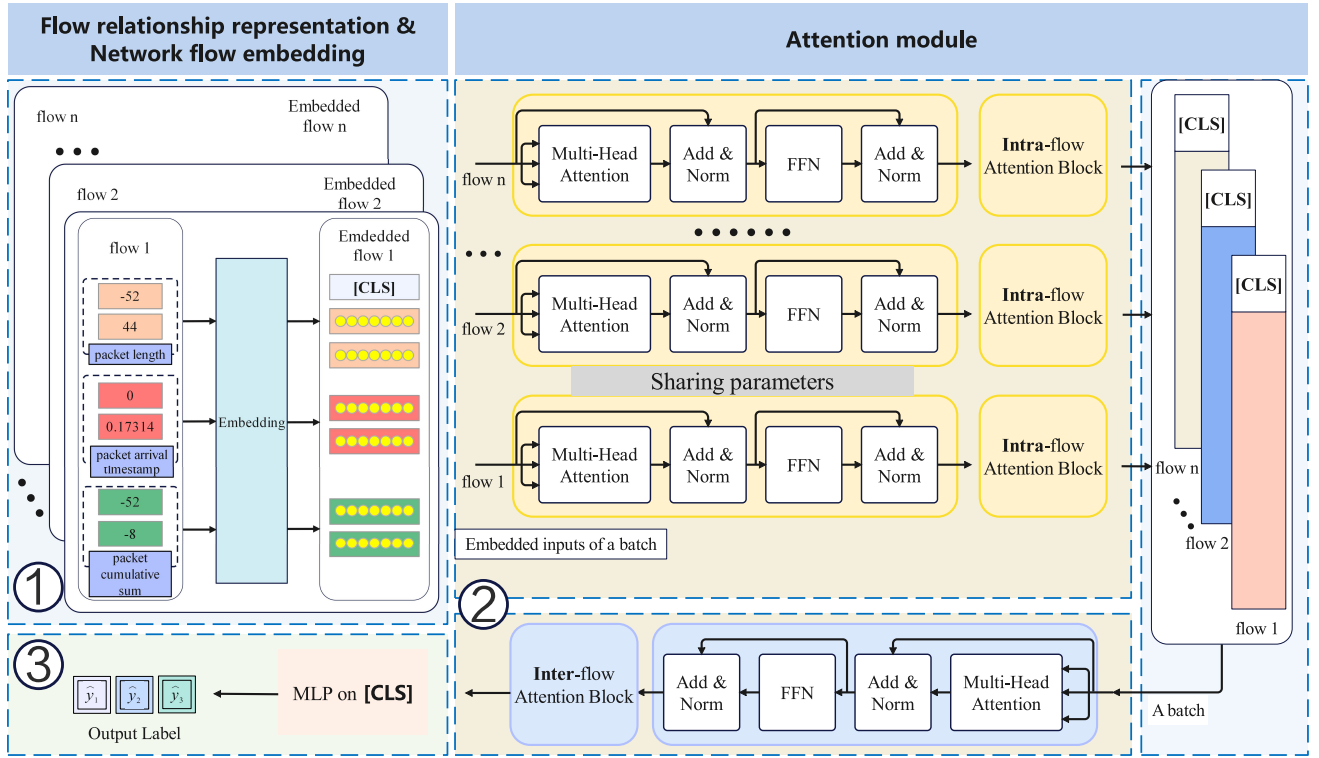


Fig. 1. System overview of the IIT model.

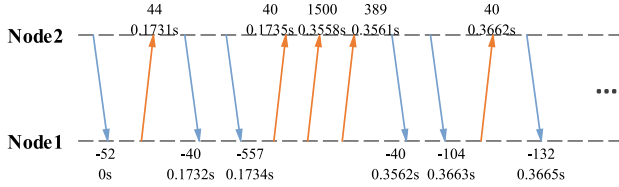


Fig. 2. The client-server interaction process.

weight to the noise. The input to the Inter-flow Attention Block consists of a batch of flow features that passed through the Intra-flow Attention Block. Finally, each network flow learns a robust feature representation that is fed into the multilayer perceptron (MLP) for correct classification.

A. Flow Relationship Representation & Network Flow Embedding

The DApps network flow representation and flow embedding is detailed in this section. The data interactions of DApps are performed through smart contracts, and when a user utilizes a particular DApp service it may be concurrently generated into multiple network flows that collaboratively provide the service, as shown in Fig. 2 [30]. From the Node1's perspective, the lengths of the uplink and downlink packets are set to positive and negative values, respectively, with each packet having a packet arrival timestamp.

In the existing literature, a network flow containing k packets is typically represented as a packet length sequence $ps = [p_1, p_2, \dots, p_k]$ [5], [16], [17]. However, representing network

flows using only a sequence of packet lengths does not completely represent the rich information contained in the network flow. Although graphs can represent richer information in a network flow, the graph construction process consumes more computational resources than vectors [15], [31], [32]. Additionally, generating a corresponding graph with network flows requires considerable preprocessing time as the size of the dataset increases, which is highly undesirable.

The majority of packets in the network flow complete transmission within the first 5 s in most different classes of DApps. Different DApps have various flow durations, but most packets of DApp network flows finish transmission before the 10th second. However, Tether still has a large number of packets in transit after the 10th second. This suggests that the packet arrival timestamp sequence for DApps is a distinguishing feature that can be used for classification.

A relatively simple structure was used in this study to represent the network flow. Here, meta information such as the packet length, direction of flow, and arrival timestamp were utilized without decrypting the packet. First, the packet length sequence of the DApp network flow, as shown in Fig. 2, is expressed as:

$$ps = [-52, 44, -40, -557, 40, 1500, 389, -40, -104, 40, -132, \dots]. \quad (1)$$

Subsequently, each packet has an arrival timestamp, and its sequence is expressed as:

$$pts = [0, 0.1731, 0.1732, 0.1734, 0.1735, 0.3558, 0.3561, 0.3562, 0.3663, 0.3662, 0.3665, \dots]. \quad (2)$$

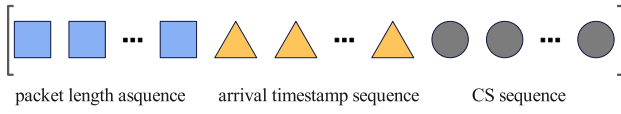


Fig. 3. The network flow representation.

The packet-length CS sequence is expressed as:

$$CS \text{ sequence} = [-52, -8, -48, -605, -565, 935, 1324, 1284, 1180, 1220, 1088, \dots]. \quad (3)$$

The DApp network flow as a vector is expressed as:

$$f = ps || pts || CS \text{ sequence} = [p_1, p_2, \dots, p_k] || [p_{t_1}, p_{t_2}, \dots, p_{t_k}] || [p_1, p_1 + p_2, \dots, p_1 + \dots + p_k], \quad (4)$$

where $len(f) = 3k$.

Only the first k packets in the flow are used if the number of packets in the DApp flow is greater than k , where k is a hyperparameter. The length of the insufficient packets is set to 0 if the number of packets in the DApp flow is less than k , and the packet arrival timestamps are also replaced by 0. Thus, it is straightforward to represent the arbitrary network flow as a vector f . The DApp network flow representation is shown in Fig. 3.

Benefits of flow relationship representation:

The vector f is more suitable for representing DApp network flows. Overall, there are three benefits to using the vector f to represent the features of DApp network flows, each of which has a significant benefit for traffic classification. These benefits are:

- 1) Vector f can extract the features of DApp traffic from four aspects, each of which has been proven to be valuable for traffic classification: packet direction, length, arrival time, and length CS information.
- 2) The vector representation of DApp network flows is simple, easy to compute, and fast, which is different from methods based on graphical representations. Graphical representations contain richer information about DApp network flows but these methods require more computational and storage resources when representing network flows, which implies that the classification speed will be relatively slow.
- 3) The representation of DApp network flows only utilizes the packet length, time of arrival, direction of transmission, and statistics of the data flow visible to the DApps-encrypted traffic as a means of inferring the specific class of DApps to which the network flow belongs. This representation does not involve information in packet-specific fields or encrypted payloads, and it can be migrated to other encrypted traffic classification tasks.

Inspired by the word embedding technique in natural language processing, the purpose of the embedding module is to accelerate model convergence by mapping each feature in the network flow f to a vector of fixed dimensions.

Inspired by BERT [33], we introduce a class token to represent the entire sample sequence, denoted as $[CLS]$, which is a learnable embedding that is added to each sample. At this point, the sample f_i can be represented as $f_i =$

$[[CLS], f_i^1, f_i^2, \dots, f_i^{3k}]$. E represents a module that embeds each feature into a d -dimensional space, using a different embedding function for each feature value in sample f_i [34]. Subsequently, for a given $f_i \in \mathbb{R}^{3k}$, we get $E(f_i) \in \mathbb{R}^{(3k+1) \cdot d}$. The embedding module can then be expressed as:

$$E(f_i) = E([CLS], f_i^1, f_i^2, \dots, f_i^{3k}) = [mlp_0([CLS]), mlp_1(f_i^1), \dots, mlp_{3k}(f_i^{3k})], \quad (5)$$

where mlp_j denotes the mapping $mlp_j : \mathbb{R} \rightarrow \mathbb{R}^d$.

B. Attention Module

The first and second sub-layers are the Intra- and Inter-flow Attention Blocks, respectively, as illustrated in Fig. 1. The attention module is described in more detail below.

Intuitively, a network protocol can be considered as a language for communication between network entities, and packets can then be viewed as sentences obeying the grammatical rules of that language. Therefore, the packet length and arrival time sequences in DApp network flows have certain temporal and semantic characteristics. The self-attention layer is one of the most important structures in a transformer network that uses residual connections followed by layer normalization. The essence is to use the similarity between the query and key vectors as the weight and perform a weighted summation on all value vectors. In this study, the encoder module in the transformer was used to efficiently extract information from different dimensions of space. That is, the intra- and inter-flow information was used to capture richer feature information through the mechanism of multi-head self-attention.

1) *Intra-Flow Attention Block*: The purpose of the Intra-flow Attention Block is to establish the relationship between any two features in a DApps network flow representation vector f_i , that is, to capture the intra-flow information. An Intra-flow Attention Block with a multi-head attention mechanism can assign various attention weights to different feature information of f_i in the vector to realize reasonable and effective feature extraction of the vector f_i .

For a given $f_i \in \mathbb{R}^{3k}$, we get $E(f_i) \in \mathbb{R}^{(3k+1) \times d}$. If only one head exists, then:

$$q = W_q \cdot E(f_i), k = W_k \cdot E(f_i), v = W_v \cdot E(f_i), \quad (6)$$

$$attn = \frac{\text{softmax}(q \cdot k^T)}{\sqrt{d}}, \quad (7)$$

where W_q , W_k , and W_v denote learned parameter matrices. It is worth noting that the attention matrix $attn$ reflects the relationship between two arbitrary elements of the input sequence f_i . The single-head attention formula can then be expressed as:

$$Attention(q, k, v) = \frac{\text{softmax}(q \cdot k^T)}{\sqrt{d}} \cdot v. \quad (8)$$

An illustration of how the intra-flow attention was performed on a single head is shown in Fig. 4.

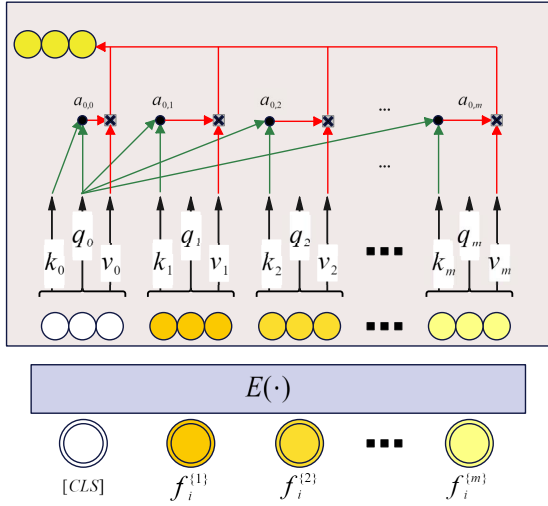


Fig. 4. Intra-flow single-head self-attention.

2) *Inter-Flow Attention Block*: An Inter-flow Attention Block was designed to focus on the relationships between different flows. The difference here is that the attention scores are computed across different flows in a given batch instead of limiting the focus to the intra-flow [35]. Unlike existing studies that focused on flow sequences, the Inter-flow Attention Block proposed in this study overcomes the temporal limitation of flow sequences. That is, the Inter-flow Attention Block can compute a temporally-similar flow sequence to obtain the relationship of each flow with other flows within the context of the current flow sequence. More importantly, given a specific flow, the Inter-flow Attention Block can break through the time constraints and allow all features from different samples to communicate with each other. An illustration of how the inter-flow attention is performed in a single head is shown in Fig. 5.

In the single-head case, for an embedded flow of a batch, we first get $(\text{batch}, 3k + 1, d) = f.\text{shape}$, which differs from the Intra-flow Attention Block in that each Embedded flow is regarded as q, k, v . The relationships between the different network flows are computed based on this and expressed as:

$$f = \text{reshape}(f, (1, \text{batch}, (3k + 1) \cdot d)), \quad (9)$$

$$q = W_q \cdot f, k = W_k \cdot f, v = W_v \cdot f, \quad (10)$$

$$\text{attn} = \text{softmax}\left(\text{mm}(q, \text{Transpose}(k, (0, 2, 1))) / \sqrt{d}\right), \quad (11)$$

$$\text{out} = \text{mm}(\text{attn}, v), \quad (12)$$

where mm denotes matrix multiplication and Transpose denotes a function that swaps the dimensions of an array.

In the multi-head case, q, k , and v are projected onto the d/h dimension instead of the d dimension, where h is the number of heads. The updated vectors v_i are then concatenated to obtain a vector of length d .

C. Classification Module

The final component of the IIT model is the classification module, which is essentially an MLP. For the final prediction step, the embedded value corresponding only to the [CLS]

Algorithm 1 Workflow of the IIT Model. For Simplicity, We Describe Just One Head

Input: The raw training dataset $D = \{(f_i, y_i)\}_{i=1}^n$, embedding size d , packet number k and batch size.

Output: The trained model IIT.

```

1: for data in enumerate(train_dataloader) do
2:   Optimizer.zero_grad()
3:   for each  $i \in [1, \text{batch}]$  do
4:      $f_i = \mathbf{E}(f_i)$ 
5:   end for
6:   /* Intra-flow Attention Block */
7:    $\text{cls} = \mathbf{E}(\text{zeros}([\text{batch}, 1, d]))$ 
8:    $f = \text{cat}((\text{cls}, f), \text{dim} = 1)$ 
9:    $f = \text{attention\_1}(f)$ 
10:   $f = \text{feedforward\_1}(f)$ 
11:  /* Inter-flow Attention Block */
12:   $\text{batch}, 3k + 1, d = f.\text{shape}$ 
13:   $f = \text{reshape}(f, (1, \text{batch}, (3k + 1) \cdot d))$ 
14:   $q, k, v = \text{mm}(W_q, f), \text{mm}(W_k, f), \text{mm}(W_v, f)$ 
15:   $\text{attn} = \text{softmax}(\text{mm}(q, \text{Transpose}(k, (0, 2, 1))) / \sqrt{d})$ 
16:   $\text{out}_1 = \text{mm}(\text{attn}, v)$ 
17:   $f = \text{feedforward\_2}(\text{out}_1)$ 
18:   $\text{out}_2 = \text{reshape}(f, (\text{batch}, (3k + 1), d))$ 
19:   $\text{representation} = \text{out}_2[:, 0, :]$ 
20:   $y_{\text{outs}} = \text{mlp}(\text{representation})$ 
21:   $\text{loss} = \text{CrossEntropyLoss}(y_{\text{outs}}, y_{\text{ground\_truth}})$ 
22:   $\text{loss.backward}()$ 
23:  Optimizer.step()
24: end for

```

class token is passed through a simple MLP with a single hidden layer and rectified linear unit activation to obtain the final output. Because the final embedding of [CLS] class token is informative about the input flow. Ultimately, the classification process is then optimized by minimizing the cross-entropy loss between the predicted distribution $\hat{y}$ and the true label y . The workflow of the IIT training phase is presented in Algorithm 1.

IV. EXPERIMENTAL ANALYSIS

The effectiveness of the IIT model for classifying DApp traffic was evaluated using real-world network-traffic data. The dataset and experimental setup used for the evaluation are described in the following sections. The hyperparameters of the IIT model were tuned, and the performance of the IIT model was compared with those of state-of-the-art models by defining suitable metrics. In addition, it was demonstrated that the IIT model exhibited excellent generalization capabilities for malicious traffic classification. Furthermore, an ablation study was conducted to determine the importance of each component in our proposed model. Finally an explanation of the model's classification behavior was given.

A. Dataset Collection

Network traffic was captured and non SSL/TLS encrypted traffic was filtered out when a user accessed DApps using

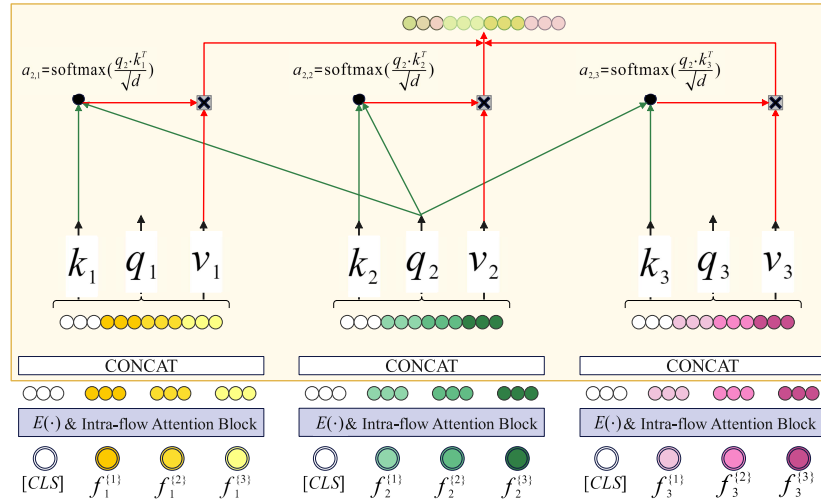


Fig. 5. Inter-flow single-head self-attention, where batch = 3 and embedding dimension $d = 3$.

TABLE II
DETAILS OF THE DAPPS DATASET

Categories	DApps (Number of Flows)	Percent (%)
Finance	AaveP (7892), BarnBridge (3608), Nexo (1018), Synlev (1043), Tether (1532), Paxos (8852), Harvest (6156), Ampleforth (2167)	12.6
Exchanges	Dydx (2215), Nash (2340), 1inch (7448), Tokenlon (7711), Curve (5005)	9.6
Marketplace	Superrare (2083), Opensea (8181), Knownorigin (6679), District0x (3951), Cryptokitties (12478)	12.9
High_risk	Gandhiji (1200), Minereum (4102), Forsage (7572), Doubleway(2794)	6.0
Governance	Aragon (3576), Kleros (1364), Decentral Games (5322)	4.0
Development	Enigma (1550), MyContract (2134), Aelf (2771)	2.5
Gambling	Kingtiger (1875), Edgeless (9639), Dice2win (3638)	5.9
Game	Nftegg (1007), Pooltogether (8241), F1deltatime(9222)	7.1
Social	2key (9601), Livepeer (5341), Catallax (1409)	6.3
Media	Avxchange (48552), Refereum (3886)	20.3
Storage	Storj (2097), Numerai (6694)	3.4
Property	Decentraland (6273)	2.4
Security	Chainlink (9722)	3.8
Identity	LikeCoin (8530)	3.3
Total	258471	100

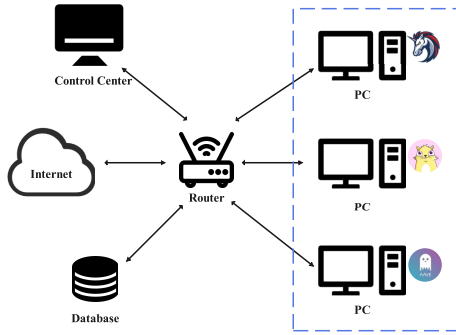


Fig. 6. Process of DApps traffic capture.

Chrome on a personal computer, as shown in Fig. 6. Bidirectional flows were considered in this study. The network packets were divided into flows according to the five-tuple. Subsequently, the flow relationship representation module was used to obtain the feature vector f corresponding to each flow, and the feature vector f and corresponding label y were imported into a comma-separated value (CSV) file.

The top 44 DApps on Ethereum with the highest number of users were selected to construct a dataset that consisted of 14 categories, including Exchanges, Finance, Games, and High_risk. A total of 258,471 flows were collected, and the number of flows for each DApp are summarized in Table II. The number of flows of different classes of DApps varied significantly, that is, there was class imbalance [36].

A public dataset of malicious traffic (Malicious_TLS) containing 92,034 network flows was used to test the generalization ability of the model [37]. This dataset contained traffic generated by 23 families of malicious code active between 2018 and 2021. All traffic was collected from a real network and encrypted using TLS.

B. Methods in Evaluation

The following eleven state-of-the-art approaches were leveraged for evaluation to fully demonstrate the performance of the IIT model.

TABLE III
PERFORMANCE OF THE STATE-OF-THE-ART METHODS

Methods	Acc (%)	Prec (%)	F1 (%)
AppScanner	84.284±0.758	97.118±0.879	90.582±0.651
GraphDApp	89.532±1.177	90.738±1.212	90.607±2.458
DF+D	69.718±0.658	72.836±0.589	61.516±3.651
DF+L	85.933±0.726	85.614±1.071	81.336±1.283
CUMUL	87.442±0.875	87.064±1.492	83.393±1.155
FS-Net	84.822±1.292	82.310±1.758	79.808±0.914
BIND	93.616±0.579	89.365±1.261	90.841±0.988
ACID	93.433±1.052	93.614±0.655	93.379±0.658
ProGraph	91.985±1.001	91.985±2.054	91.982±1.262
RoFi	92.415±1.512	96.753±2.125	88.914±0.989
FastTraffic	89.101±0.695	89.464±1.127	89.051±0.995
IIT	97.164±0.928	97.275±1.161	97.241±1.051

AppScanner [5] uses 54 statistical features of mobile app network flows as app fingerprints, which are used as inputs to an RF to realize real-time identification of mobile app network flows.

GraphDApp [15] transforms the packet length sequence of a DApps network flow into a traffic interaction graph, converts the DApps network flow classification problem into a graph classification problem, and designs a graph neural network-based classifier using MLP.

DF + D [16] uses packet direction sequences as inputs to CNNs to identify traffic accessing different websites via Tor.

DF + L [16] uses the same neural network structure as DF+D. However, the input is a sequence of packet lengths used to classify different Web traffic.

FS-Net [17] uses packet-length sequences of encrypted traffic as input and classifies SSL/TLS traffic by learning the feature representation of the flow sequences through a multi-layer bi-GRU encoder and decoder.

CUMUL [7] abstracts the loading process of a webpage by generating a cumulative behavioral representation of its trace, such as packet ordering or burst behavior, which is fed to an RF classifier.

BIND [6] extracts characteristics from encrypted traffic by utilizing data dependencies that occur during the sequential transmissions of network packets. RF is then used to identify the website and app fingerprinting.

ACID [19] relies on low-dimensional embeddings learned using a lightweight neural model comprised of multiple kernel networks that optimally separate samples of different classes to classify malware traffic using an adaptive clustering network.

ProGraph [22] proposes a graph-based approach that uses destination, certificate, and domain features to ensure robust network traffic classification in various network environments.

RoFi [23] uses Traffic Aggregation Matrix (TAM) to represent network flows which is a novel attack model for website fingerprinting.

FastTraffic [38] uses the raw bytes in the packet as input and uses only three layers of MLP to accomplish fast classification.

Three widely used metrics were employed to evaluate the performance of IIT: Accuracy (Acc), Precision (Prec) and F1-Score (F1). We divided the dataset into training, validation and testing in the ratio of 8:1:1. We resort to 10-fold

cross-validation and obtain the average and standard deviation of metrics to measure the performance of IIT. All experiments were carried out on the same platform equipped with an Intel 12700KF@3.60GHz, 32GB RAM, and an NVIDIA GeForce RTX 3070Ti.

C. Evaluation on DApps

The evaluation of the effectiveness and efficiency of the IIT model is presented in this section. The dataset was essentially a multi-classification problem where each DApp was a label, as listed in Table II. The goal was to deduce the DApp to which the flow belonged.

1) *Effectiveness*: The Acc, Prec, and F1 were utilized as metrics to measure the performances of all methods. The weighted Acc, Prec, and F1 of the ten models are listed in Table III.

The following key observations can be deduced from Table III.

(1) The IIT model outperformed all other methods with the highest Acc (97.164%), Prec (97.275%), and F1 (97.241%). AppScanner achieved the second-highest Prec (97.318%), but its Acc was only 84.284% because it classified well in some categories but not in others. A total of 19 DApps had Recall values of less than 0.8. For example, the classification effect was the worst for Gandhiji, Harvest, Livepeer, nexo, and nftegg, whose Recall values were less than 0.6. However, the proportion of the number of flows accounted for by these five DApps reached 5.7%.

(2) Traditional classifiers based on machine-learning methods, such as AppScanner and BIND, obtained relatively good results for DApps classification, and AppScanner outperformed DF+D. Compared with DF+D, DF+L used packet length sequences as inputs to the CNN-based classifier. DF+L outperformed DF+D, indicating that packet length sequences were a more effective flow representation than packet direction sequences for the DApps classification task. In addition, this proved that the flow representation in this study was effective.

(3) FS-Net and GraphDApp did not perform as well as the proposed IIT model. Although GraphDApp represented the flows by constructing a traffic interaction graph (TIG) with rich information, different DApps used the same cryptographic protocols that were deployed on the same blockchain platform.

TABLE IV
PERFORMANCE OF STATE-OF-THE-ART METHODS ON COARSE-GRAINED DAPPS

Category	Finance		Gambling		Game		High_risk	
Methods	Acc (%)	F1 (%)	Acc (%)	F1 (%)	Acc (%)	F1 (%)	Acc (%)	F1 (%)
AppScanner	78.195±0.751	87.743±1.649	76.705±0.788	86.941±0.570	68.212±1.346	81.485±0.175	82.241±1.697	90.128±0.695
GraphDApp	91.101±0.124	92.237±0.457	93.473±0.121	93.051±0.849	87.672±0.587	88.820±0.745	94.106±0.114	95.217±0.695
DF+D	74.001±0.245	65.144±0.547	61.093±1.598	62.197±0.744	78.624±2.010	70.795±1.474	72.248±2.157	78.223±0.113
DF+L	92.182±1.245	82.283±1.220	63.934±1.451	77.374±0.565	86.689±0.153	83.197±0.665	93.601±0.254	91.411±0.546
CUMUL	89.679±1.894	80.116±1.454	82.250±1.645	82.363±1.454	82.398±2.416	81.369±2.451	88.651±2.546	91.661±0.451
FS-Net	83.202±0.416	80.128±0.158	83.228±2.144	79.536±0.562	86.933±0.561	75.911±0.997	89.811±0.654	86.767±0.486
BIND	94.994±2.145	95.030±1.641	91.562±1.648	92.159±1.161	96.987±0.541	93.103±0.155	99.205±0.854	97.121±0.564
ACID	94.988±1.496	92.141±1.160	92.153±1.614	92.172±0.561	92.022±1.654	91.843±2.151	96.500±1.888	96.296±1.487
ProGraph	92.683±1.564	93.878±1.455	91.139±0.615	91.139±2.155	98.000±2.455	94.231±1.615	99.048±2.165	97.758±1.898
RoFi	92.165±0.985	91.545±0.956	93.651±0.848	91.541±1.554	92.465±1.125	92.566±1.452	93.515±1.125	93.115±2.046
FastTraffic	90.651±0.845	91.165±1.125	91.155±0.647	92.131±1.237	90.481±0.984	89.984±1.382	90.114±1.523	90.482±1.834
IIT	97.944±0.959	95.414±1.222	95.649±1.156	93.105±1.166	95.976±1.560	94.775±1.565	97.600±1.141	98.308±0.841

Consequently, the similar handshaking process and data interactions lead to very similar TIGs being constructed by different DApps flows. Consequently, it was difficult to achieve better classification accuracy. FS-Net lacked the learning of inter-flow relationships even when it achieved a classification accuracy of 84.822%. Conversely, IIT learned intra- and inter-flow relationships, and therefore performed better.

An experiment was designed in which the labels of all the flows belonging to the DApps in a broad category were set to the same labels. This was done because there was a great similarity in the behavioral patterns of DApps in the categories of Finance, Gambling, Games, and High_risk [39]. There were a total of 14 broad categories of DApps, as listed in Table II. The experimental results are listed in Table IV. The IIT model was optimal for classifying the Finance and Gambling classes of DApps. The clustering-based ProGraph method achieved 98% accuracy in recognizing the DApps of the Game class, and the machine learning-based BIND method achieved 99.205% accuracy in recognizing the DApps of the High_risk class. Overall, IIT achieved the best classification results for broad category DApps.

2) *Efficiency*: The testing times of all methods were compared after being trained on the DApps dataset to test the actual detection speed of each method. The concepts of feature extraction time (FET) and classifier prediction time (CPT) were used in GraphDApp [15]. Prediction time refers to the time required to discriminate each flow, and total time refers to the sum of the FET and CPT. The testing times for all methods are listed in Table V.

GraphDApp exhibited the longest FET because building a flow into a TIG is an extremely time-consuming process that requires operations to add edges and nodes. The FET of AppScanner was the second longest because of the need to utilize the packet length to compute the 54-dimension statistical features. The FET for the IIT model was slightly more complex than those of the DF+D and DF+L because it needed to compute the cumulative sum of packet length sequences, which is a process used to compute the statistical characteristics of the packet length. Among all the deep-learning-based methods, ProGraph exhibited the longest CPT because of its complex model structure. DF+D shares the same model structure as DF+L, with only the inputs different. Therefore, these classifiers had the same prediction time.

TABLE V
AVERAGE TESTING TIMES OF DIFFERENT METHODS

Methods	FET (ms)	CPT (ms)	Total time (ms)
AppScanner (cpu)	1.60	0.08	1.68
GraphDApp	21.10	0.15	21.25
DF+D	0.11	0.17	0.28
DF+L	0.12	0.17	0.29
CUMUL (cpu)	0.05	6.12	6.17
FS-Net	0.16	0.47	0.63
BIND (cpu)	0.16	1.39	1.55
ACID (cpu)	0.15	0.66	0.81
ProGraph	0.21	46.20	46.41
RoFi	0.27	0.32	0.59
FastTraffic	0.13	0.11	0.24
IIT	0.21	0.53	0.74

However, FastTraffic had the smallest CPT since it uses only three layers of MLP for traffic prediction.

Overall, the maximum and minimum total times used to predict an unknown flow of DApps across all methods were 21.25 and 0.28 ms, respectively. The CNN-based models (DF+D and DF+L) had similar total times. GraphDApp had the largest total time owing to its time-consuming FET. FS-Net had the second-longest prediction time owing to its complex structure, and AppScanner had the third-longest total time because it needed to compute 54-dimensional statistical features. The FET for the IIT model only involved calculating the CS of the packet length sequences and writing them to a CSV file, which was accomplished in only 0.74 ms. This was the highest accuracy discrimination for unlabeled DApps-encrypted traffic.

3) *Summary*: This section presents a summary of the experimental results on the effectiveness and efficiency of all methods tested on the DApps dataset. It was verified that the proposed model completed the classification of an unlabeled flow of DApps in the shortest possible time with the most accurate identification results. Here, t-SNE [40] is used to provide a more intuitive understanding of the classification results and visualize the ability of the IIT model, as shown in Fig. 7.

At the fifth epoch, most DApps were mixed together and difficult to distinguish. At the tenth epoch, DApps of different

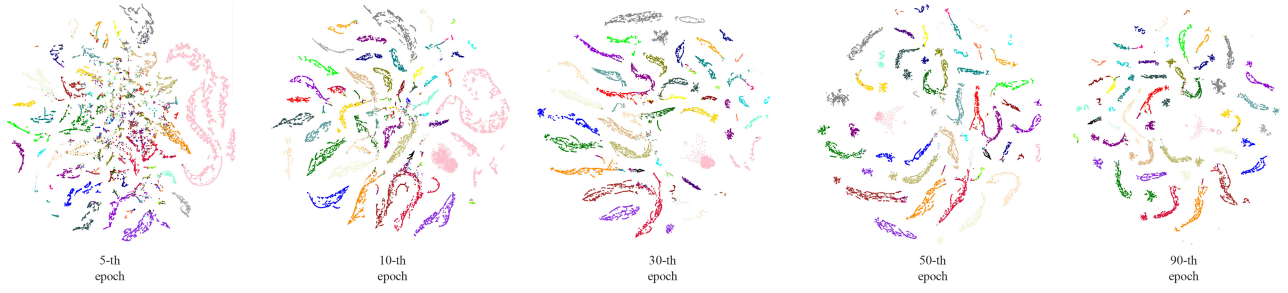


Fig. 7. Visualization to demonstrate the performance of the IIT model using t-SNE.

TABLE VI
PARAMETER SELECTIONS

Parameters	Search Space	Selected Value
Optimizer	[Adamw, Adam, SGD]	Adamw
Learning Rate	[0.00001, ..., 0.0005]	0.00005
Batchsize	[16, ..., 256]	32
Attention_dropout	[0, ..., 0.9]	0.8
FeedForward_dropout	[0, ..., 0.9]	0.8
Activation Functions	[Tanh, ReLU, ELU]	ReLU
Number of FC Layers	[1, ..., 4]	2
Epochs	[50, ..., 200]	100
k	[25, ..., 45]	40
L	[1, 2, 3]	1
(h, d)	[(1, 4), ..., (16, 16)]	(4, 8)

classes gradually moved away from one other, and those of the same class gradually came closer together. The classification results improved as the number of epochs increased when epoch > 30 .

D. Parameter Tuning

Determining the optimal hyperparameters for the IIT model is discussed in this section.

1) *Parameters Selection*: The most important step in training transformer-based neural network models is parameter tuning, which aims to trade-off the model complexity, classification accuracy, and speed to obtain an optimal set of hyperparameters. Finding the optimal set of hyperparameters is a challenging task owing to the extensive number of IIT hyperparameters and large training datasets. Therefore, an optimal set of hyperparameters was selected from an interval. Specifically, for a fixed hyperparameter, a larger interval was first considered and then it was determined whether the hyperparameter should be increased or decreased according to the model performance. Then, small intervals for each hyperparameter were considered that improved the performance of the model. Finally, the combination of values in these small intervals constituted the candidate set of optimal hyperparameters. Accuracy was used as the evaluation metric.

The hyperparameter candidate intervals and final selected values are listed in Table VI. A learning rate of 0.00005, batch size of 32, 100 epochs, and a loss function for cross-entropy were finally chosen for the IIT model.

TABLE VII
EFFECTS OF THE NUMBER OF LAYERS L ON ACCURACY
WHEN $h = 4, d = 8$

L	1	2	3
Acc(%)	97.164	97.104	96.714

The effect of other important hyperparameters on model complexity and accuracy were also investigated, as outlined below.

2) *Depth and Attention Heads: Embedding Size*: The IIT training process involved the following hyperparameters:

L , which denotes the number of layers in the attention module. The depth of the neural network model is critical for classification, and intuitively increasing the depth of the model improves classification accuracy. In the IIT model, the model depth was increased by continuously overlaying one Attention module on top of another.

h , which denotes attention heads in the multi-head self-attention mechanism. This hyperparameter allows the model to use multiple self-attention sub-layers in the same layer with different query, key, and value projections in each sub-layer. This allows the model to focus on different primitive features separately in different representation subspaces.

d , which denotes the embedding dimensions of each feature. Previously, each one-dimensional feature was mapped to a d -dimensional vector. The experiments proved that the parameter d affected the model classification accuracy. In the case of the multi-head self-attention mechanism, the Inter-flow Attention Block projects q, k, v onto the d/h dimension instead of a given dimension d , where h denotes the number of heads. All updated vectors v_i are then concatenated to obtain a vector of length d . This requires that $h|d$, that is, h can only be a factor of d when the embedding dimension d of the feature is fixed.

The experimental results for different values of the above hyperparameters are presented in the following section.

First, the effect of the number of Attention module layers L on the classification accuracy of the model is discussed. The model classification accuracy for a fixed set of h and d decreased when L increased, as shown in Table VII. The model will be more complex if the number of stacked layers L is larger. Consequently, the model training time will be longer. Therefore, 1 was chosen as the value of L .

In the following, $L = 1$ was fixed and the effects of the attention heads (h) and embedding size (d) on the model

TABLE VIII
EFFECT OF d AND h ON ACCURACY(%), WHERE
 h MUST BE A FACTOR OF d

	$d = 4$	$d = 8$	$d = 12$	$d = 16$
$h = 1$	95.822	96.945	96.857	97.164
$h = 2$	94.121	96.569	96.693	97.220
$h = 3$	-	-	96.777	-
$h = 4$	96.111	97.164	96.577	97.064
$h = 6$	-	-	96.677	
$h = 8$	-	96.845	-	97.208
$h = 12$	-	-	96.613	-
$h = 16$	-	-	-	97.761

TABLE IX
EFFECTS OF THE NUMBER OF PACKETS k ON ACC WHEN
 $L = 1, h = 4$ AND $d = 8$

Packet number (k)	45	40	35	30	25
Acc (%)	96.741	97.164	96.473	96.445	96.222
$Param \times 1e6$	20.6	16.4	12.7	9.5	6.8

classification performance were studied. The h and d values exhibited a significant impact on model performance for a fixed $L = 1$. The highest classification accuracies occurred at $(h, d) = (4, 4)$, $(4, 8)$, $(3, 12)$, and $(16, 16)$, which were 96.111%, 97.164%, 96.777%, and 97.761%, respectively, as listed in Table VIII. It is worth noting that the model parameters increased as d and h increased. The accuracy improved by 1% for $(h, d) = (4, 8)$ compared with that of $(h, d) = (4, 4)$. However, the increase in the model parameters did not result in a large performance improvement if (h, d) was $(3, 12)$ or $(2, 16)$.

In addition, the classification accuracy of the model decreased if the h and d values were very small. This suggests that an appropriate number of heads and embedding dimensions are crucial for model performance in a multi-head self-attention mechanism. In summary, $h = 4$ and $d = 8$ were selected as the final hyperparameter settings.

3) *Packet Number*: The first k packets of the TLS flow were used for DApp traffic classification in the IIT model. It should be noted that the hyperparameter k will have a significant effect on the model classification performance, as shown in Table IX. First, the IIT accuracy values generally decreased for lower values of k . Second, the time overhead of the DApps classification model increased with the number of packets considered per DApp flow. An interesting phenomenon is that the accuracy of IIT also decreased if the value of k was too high. This was because the hyperparameters used in the experiment at this point were $L = 1$, $h = 4$, and $d = 8$, and an increase in the number of packets led to an increase in the dimensions of the feature vectors. Additionally, the number of attention heads in the multi-head self-attention mechanism was not sufficient enough to pay attention to the different original

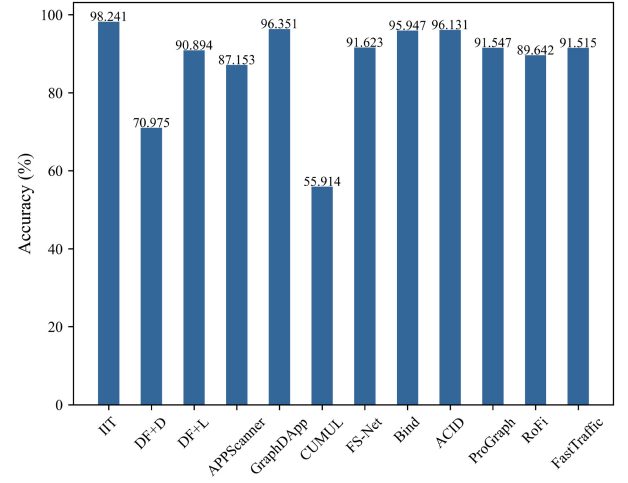


Fig. 8. Accuracy of different models on Malicious_TLS dataset.

features in each of the representational subspaces in the high-dimensional vectors.

In addition to this, the number of model parameters is quadratically related to the model input dimension, with a larger model input dimension leading to a dramatic increase in the number of model parameters. For example, when $k = 25$, the number of model parameters is 6.8M, and when $k = 45$, the number of model parameters reaches 20.6M. The proposed model achieved acceptable accuracy for the 44 class classification. Only the first 40 packets of the flow were used to avoid introducing more packets into the input features, which would make the model highly complex for trivial performance gains.

E. Evaluation on Malicious Traffic Classification

Although the IIT model was designed to identify specific DApps, its powerful feature extraction capabilities were also applicable for classifying malicious traffic. Experiments were conducted using a publicly available Malicious_TLS dataset to evaluate the generalization ability of the IIT model for malicious traffic classification. The Malicious_TLS dataset contains 92,034 flows with 23 classes of malicious traffic, such as burpsuite_scan and nessus_scan. It should be noted that only packet-length sequences were used from the dataset. The classification results of the different methods are shown in Fig. 8.

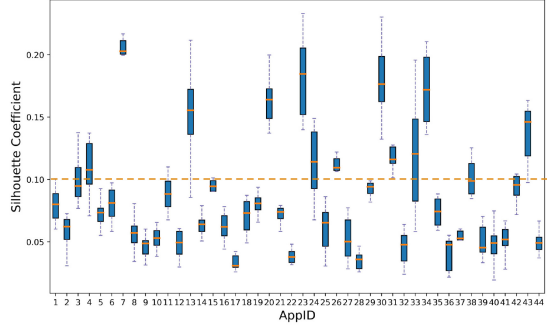
GraphDApp, BIND, and ACID exhibited classification accuracies greater than 95%, indicating that their extracted features were sufficient for identifying malicious traffic. Compared with the results listed in Table III, the accuracy of CUMUL decreased by 30% because most of the malicious traffic flows were short and contained only a few packets, and it is difficult for CUMUL to extract discriminative features. The accuracy of the IIT model exceeded 98%, outperforming all other methods, indicating that it could also be effectively applied to malicious traffic classification.

F. Ablation Studies

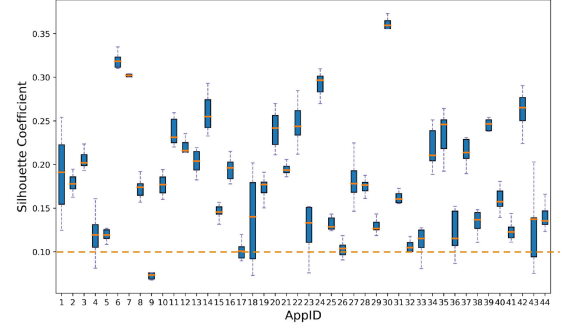
The classification results were also affected by the structure of the neural network and model input sequence, such as the

TABLE X
IMPACT OF DIFFERENT FEATURES AS WELL AS DIFFERENT MODULES ON IIT PERFORMANCE

	IIT		intra-flow		inter-flow	
	Acc (%)	F1 (%)	Acc (%)	F1 (%)	Acc (%)	F1 (%)
Full	97.164	97.248	93.598/3.6↓	92.958/4.3↓	92.611/4.6↓	92.664/4.6↓
w/o packet length	95.750	94.412	95.491/0.3↓	95.455/1.0↑	89.831/5.9↓	89.621/4.8↓
w/o timestamp	92.655	92.622	92.360/0.3↓	92.276/0.3↓	85.083/7.6↓	84.957/7.6↓
w/o CS sequence	96.058	96.030	96.054/0.0↓	96.026/0.0↓	89.416/6.6↓	89.281/6.7↓



(a) Silhouette coefficient with learned intra-flow features.



(b) Silhouette coefficient with learned intra-flow and inter-flow features.

Fig. 9. Similarity measurements with silhouette coefficient metrics for different intra-flow and inter-flow features among 44 DApps. The orange line denotes the separating line for silhouette coefficient = 0.1.

Intra- and Inter-flow Attention Blocks. Therefore, the impact of the structure is discussed in detail in this section. All the experiments were conducted using the DApps dataset.

The ablation results are presented in Table X. Intra-flow and inter-flow indicate that the Attention modules of the IIT contained only the Intra- and Inter-flow Attention Blocks, respectively. The IIT model performed the best, regardless of the model input sequence. Compared with the IIT model, the intra-flow and inter-flow models showed different degrees of performance degradation. Specifically, the inter-flow model performed the worst. It is worth noting that the performance of the three models declined the most if the model input sequences were stripped of temporal information, suggesting that the packet arrival timestamp sequences are important pieces of information.

Quantitative measures were used to provide a more intuitive understanding of the Intra- and Inter-flow Attention Blocks. An ideal intra-flow representation ensured a flow similar to that from an identical DApp but different from those of different DApps. The silhouette coefficient was employed as a similarity metric [41]. The silhouette coefficient simultaneously considers the inter- and intra-class distances, in contrast to other metrics (such as the Euclidean distance) that evaluate the two separately. The silhouette coefficient was calculated for each sample point i as $sample_i = \frac{b_i - a_i}{\max(a_i, b_i)}$, where a_i denotes the average distance from sample point i to other sample points in the same class (closeness), and b_i denotes the minimum value of the average distance from sample point i to all sample points in the other classes (separateness). $sample_i$ takes a value in the range of $[-1, 1]$, and it is important to note that the closer the silhouette coefficient is to 1, the better the classification

result. A point is more likely to be assigned to the wrong label if the silhouette coefficient is less than 0. The silhouette measurements are shown in Fig. 9.

The average silhouette coefficient for most of the 44 DApps was less than 0.1 when only the Intra-flow Attention Block was used. The average silhouette coefficient for more than 90% of the DApps was greater than 0.1 when the Intra- and Inter-flow Attention Blocks were used, which was an obvious improvement.

The quantitative measures showed that the learned intra-flow and inter-flow features displayed more separation for flows from different DApps and more similarity for flows from the same DApp. Thus, the proposed IIT model achieved a considerable performance improvement.

G. Understanding Classifier Behavior

The design of the Attention module in the IIT model could easily obtain the importance of different features in the flow vector f [42]. The behavior of the proposed model using the attention output of the IIT is discussed in this section.

As described in the Attention Module, the outputs of the Intra- and Inter-flow Attention Blocks were similar to the weighted adaptation of the input features. The attention weights estimated the degree of importance of each feature in the classification task. There were two types of attention encoders in the IIT. The output of the Intra-flow Attention Block was the weighted adaptation of the hidden representations of all features in the flow, which measured the importance of each feature for the classification task. The Inter-flow Attention Block determines the relevance and

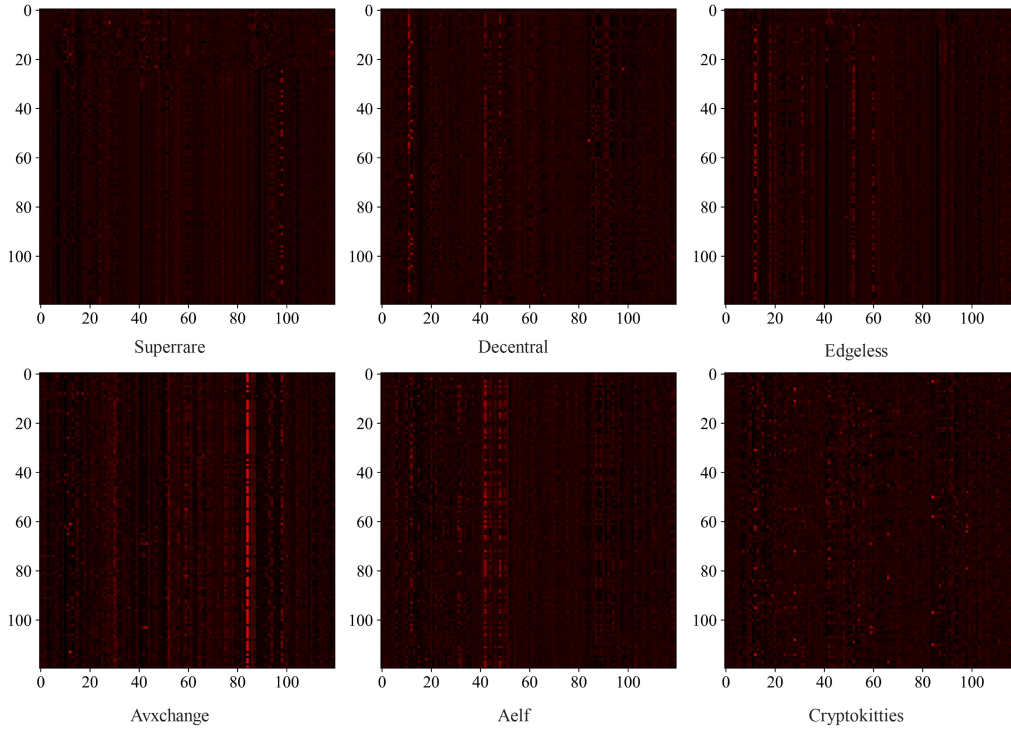


Fig. 10. Attention map of the IIT model on different DApps.

importance between arbitrary flows, obtaining a comparison of the flow with flows of the same and different classes. Thus, the importance of different features within the flow could be determined.

Typical cases that explain the IIT learned information are shown in Fig. 10. Each image is 120×120 px and represents 120 samples. Each sample is represented by a 120 dimensional vector. The background color indicates the importance of the features, and each pixel point in the image represents a feature. The redder the color, the more important the feature, and the darker the color, the less important the feature. The results demonstrated that the proposed IIT model could successfully focus on important features and assign them greater weights.

Overall, it can be seen from these six graphs that the IIT model exhibited different focuses for different classes of DApps. IIT exhibited the worst classification effect for Superrare, with only a 90.62% F1 score. In addition, the features of the package length CS sequence of the tenth to twentieth dimensions were the most important features. A better classification performance was exhibited for Cryptokitties, with a 99.23% F1 score. Furthermore, the IIT model paid attention to the features of the packet length, arrival timestamp, and length CS sequences, which were significantly different from the rest of the DApps. These three types of features of Cryptokitties exhibited a strong distinguishing ability. For Decentral and Ddgeless, the tenth to twentieth features of the packet length sequence were more important owing to the captured DApps traffic being TLS traffic. In addition, the interaction process of the first ten packets were regarded as a TLS handshaking process, thus the handshaking processes for

different DApps deployed on the same blockchain platform showed great similarity. The packet lengths of different DApps were very different from the tenth to the twentieth packet. Therefore, the IIT model paid more attention to these distinguishing features. The first ten features of the packet arrival timestamp sequence were more important for Aelf, which explains why the removal of this class of features from the packet arrival timestamp sequence in the ablation experiments resulted in a more dramatic model decay.

The proposed method focused on the most distinguishing features of different DApps. The IIT model provided a more complementary perspective on treating intra- and inter-flow relationships as complementary to each other to fully exploit their relationships.

V. DISCUSSION

Our approach currently has two limitations, the first one is that model training consumes large GPU memory, which limits efficient deployment on low-resource devices. We can appropriately reduce the model's GPU memory through model quantization without degrading the model's performance. The second limitation is the class imbalance problem. Fig. 11 shows the confusion matrix of IIT. It can be seen that 8 classes of DApps have a classification accuracy of less than 90%, among which Ampleforth has the worst classification result with an accuracy of 76.70%, while its total sample size is only 2,167 flows. We can solve the class imbalance problem by using a combination of resampling techniques or synthetic sample generation methods, which in turn improves the performance and usefulness of the overall model.

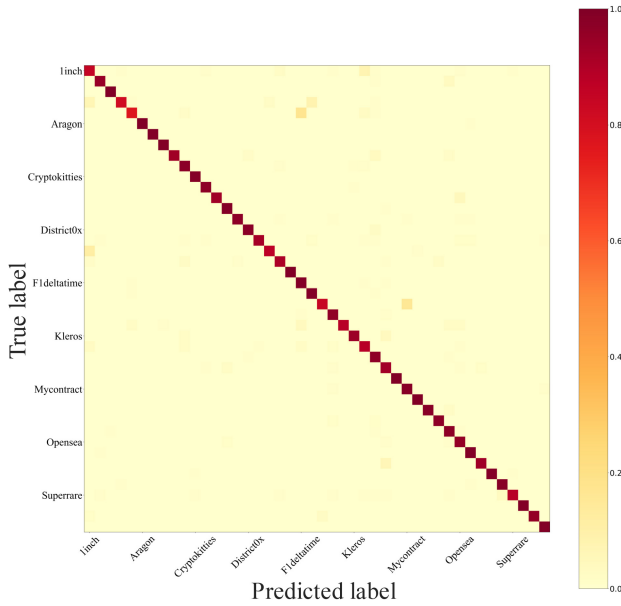


Fig. 11. Confusion Matrix of IIT.

VI. CONCLUSION

In this study, a novel encrypted traffic classification model IIT was proposed to heterogeneously mine the potential features of intra- and inter-flows for classifying DApps. Compared with existing encrypted traffic classification methods, the proposed model did not rely on expert experience to manually design features, and it utilized two encoders, Intra- and Inter-flow Attention Blocks, to mine intra- and inter-flow relationships, respectively, for effective potential feature extraction. Based on the results of the DApps dataset, it is demonstrated that the accuracy and efficiency of IIT is significantly improved over the state-of-the-art methods. In addition, experiments demonstrate that it also shows excellent generalization ability on the malicious traffic classification task.

REFERENCES

- [1] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends," in *Proc. IEEE Int. Congr. Big Data (BigData Congr.)*, 2017, pp. 557–564.
- [2] C. Lin, D. He, X. Huang, M. K. Khan, and K.-K. R. Choo, "DCAP: A secure and efficient decentralized conditional anonymous payment system based on blockchain," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2440–2452, 2020.
- [3] L. Su et al., "Evil under the sun: Understanding and discovering attacks on Ethereum decentralized applications," in *Proc. 30th USENIX Security Symp. (USENIX Security)*, 2021, pp. 1307–1324.
- [4] G. Aceto, D. Ciunzio, A. Montieri, V. Persico, and A. Pescapé, "AI-powered Internet traffic classification: Past, present, and future," *IEEE Commun. Mag.*, vol. 62, no. 9, pp. 168–175, Sep. 2024.
- [5] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Security Privacy (EuroS P)*, pp. 439–454, 2016.
- [6] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Security Appl.*, 2016, pp. 177–188.
- [7] A. Panchenko et al., "Website fingerprinting at Internet scale," in *Proc. NDSS*, 2016, pp. 1–15.
- [8] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, 2023, pp. 2066–2075.
- [9] J. Li et al., "Packet-level open-world app fingerprinting on wireless traffic," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2022.
- [10] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Disc. Data Min.*, 2022, pp. 2576–2584.
- [11] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 241–252, Feb. 2022.
- [12] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–15.
- [13] S. Rezaei, B. Kroencke, and X. Liu, "Large-scale mobile app identification using deep learning," *IEEE Access*, vol. 8, pp. 348–362, 2019.
- [14] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.
- [15] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate Decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [16] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 1928–1943.
- [17] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2019, pp. 1171–1179.
- [18] Y. Wang, G. Xiong, C. Liu, Z. Li, M. Cui, and G. Gou, "CQNet: A clustering-based quadruplet network for decentralized application classification via encrypted traffic," in *Proc. Mach. Learn. Knowl. Disc. Databases. Appl. Data Sci. Track Eur. Conf.*, 2021, pp. 518–534.
- [19] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2021, pp. 1–10.
- [20] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," 2018, *arXiv:1802.10215*.
- [21] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.
- [22] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
- [23] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 607–624.
- [24] X. Zhou et al., "CapsuleFormer: A capsule and transformer combined model for Decentralized application encrypted traffic classification," in *Proc. 19th ACM ASIA Conf. Comput. Commun. Security*, 2024, pp. 1–12.
- [25] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep./Oct. 2022.
- [26] M. Jiang et al., "FA-Net: More accurate encrypted network traffic classification based on burst with self-attention," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2023, pp. 1–10.
- [27] Y. Wang, Z. Li, G. Gou, G. Xiong, C. Wang, and Z. Li, "Identifying DApps and user behaviors on Ethereum via encrypted traffic," in *Proc. 16th EAI Int. Conf. Security Privacy Commun. Netw.*, 2020, pp. 62–83.
- [28] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, and Y. Liu, "Encrypted traffic classification of decentralized applications on Ethereum using feature fusion," in *Proc. Int. Symp. Qual. Service*, 2019, pp. 1–10.
- [29] X. Hu et al., "Identifying Ethereum traffic based on an active node library and DEVp2p features," *Future Gener. Comput. Syst.*, vol. 132, pp. 162–177, Jul. 2022.
- [30] M. Korczyński and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2014, pp. 781–789.
- [31] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.
- [32] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Towards early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.

- [33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [34] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "TabTransformer: Tabular data modeling using contextual embeddings," 2020, *arXiv:2012.06678*.
- [35] G. Somepalli, M. Goldblum, A. Schwarzschild, C. B. Bruss, and T. Goldstein, "SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training," 2021, *arXiv:2106.01342*.
- [36] M. A. S. Saber, M. Ghorbani, A. Bayati, K.-K. Nguyen, and M. Cheriet, "Online data center traffic classification based on inter-flow correlations," *IEEE Access*, vol. 8, pp. 60401–60416, 2020.
- [37] Q. Yuan et al., "BoAu: Malicious traffic detection with noise labels based on boundary augmentation," *Comput. Security*, vol. 131, Aug. 2023, Art. no. 103300.
- [38] Y. Xu, J. Cao, K. Song, Q. Xiang, and G. Cheng, "FastTraffic: A lightweight method for encrypted traffic fast classification," *Comput. Netw.*, vol. 235, Nov. 2023, Art. no. 109965.
- [39] T. Min and W. Cai, "Portrait of decentralized application users: An overview based on large-scale Ethereum data," *CCF Trans. Pervasive Comput. Interact.*, vol. 4, no. 2, pp. 124–141, 2022.
- [40] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.
- [41] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987.
- [42] A. Nascita, A. Montieri, G. Aceto, D. Ciunzio, V. Persico, and A. Pescapé, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.

Qianwei Meng is currently pursuing the Ph.D. degree with the Henan Key Laboratory of Network Cryptography Technology, Information Engineering University. His research interests include network security and unknown traffic detection.

Qingjun Yuan was born in 1993. He received the Ph.D. degree in cyber security from Information Engineering University in 2023. He is currently a Postdoctoral Researcher with the MoE Key Laboratory for Intelligent Networks and Network Security, Xi'an Jiaotong University, and also a Lecturer with the Henan Key Laboratory of Network Cryptography Technology, Information Engineering University. His research interests include side-channel analysis and encrypted traffic analytics.

Weina Niu (Senior Member, IEEE) received the Ph.D. degree in computer software and theory from the University of Electronic Science and Technology of China in 2018, where she is currently an Associate Professor with the School of Computer Science and Engineering. Her research interests include malware analysis, network attack detection, and data security.

Yongjuan Wang received the Ph.D. degree from Information Engineering University in 2009, where she currently works with the Henan Key Laboratory of Network Cryptography Technology. Her main research interests include cryptographic analysis and cyberspace security.

Siqi Lu received the Ph.D. degree from the Henan Key Laboratory of Network Cryptography Technology, Ministry of Education, where he is a Lecturer. His research interests include blockchain security, formal methods, security protocol, and big data security.

Guangsong Li received the Ph.D. degree from the Henan Key Laboratory of Network Cryptography Technology, Ministry of Education, where he is a Professor. His research interests include blockchain security, security protocol, information security, and formal methods.

Xiangbin Wang is currently pursuing the Ph.D. degree with the Henan Key Laboratory of Network Cryptography Technology, Ministry of Education. His research interests include network traffic classification and artificial intelligence.

Wenqi He received the M.S. degree from Zhengzhou University, China, in 2020. She currently works with the Henan Key Laboratory of Network Cryptography Technology, Ministry of Education. Her research interests include area of network security and complex data analysis.